



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



**НГТУ
НЭТИ** | **Факультет прикладной
математики и информатики**

Кафедра теоретической и прикладной информатики
Лабораторная работа № 4
по дисциплине «Технология разработки программного обеспечения»

Место для ввода
текста.

Группа ПМИ-33

ЕРОЩЕНКО АРТЁМ

КАУФМАН ЯНА

КОВАЛЕВА МАРИЯ

Преподаватели

ЗАЙЦЕВ М.Г.

Новосибирск, 2026

1. Цель:

Целями данного лабораторного практикума является формирование практических навыков:

- проектирования программ в технологии «абстрактных типов данных»;
- реализации абстрактных типов данных с помощью классов C++;
- использования библиотеки визуальных компонентов для построения интерфейса.

2. Задание:

1. Разработайте Универсальный калькулятор с интерфейсом в стиле Windows, который позволил бы вычислять выражения с действительными числами, простыми дробями, комплексными числами, представленными в системе счисления с основанием p .
2. Калькулятор необходимо снабдить системой справочной.
3. Для установки калькулятора необходимо создать инсталлятор.

3. Анализ задания:

Разработанная программа была разбита на 11 модулей:

1. **Enums.cs** – модуль перечислений программы. Определяет все ключевые типы перечислений, используемые для управления логикой приложения: CalculatorMode (режимы работы: PNumber, Fraction, Complex), CalculatorState (состояния конечного автомата: Start, Editing, OperationSet и т.д.), CalculatorCommand (полный набор команд: цифры, операции, функции, работа с памятью и буфером обмена), Operation (типы арифметических операций) и Function (типы однооперандных функций).
2. **TANumber.cs** – модуль абстрактного базового типа данных для чисел. Содержит абстрактный класс TANumber, который является вершиной иерархии числовых типов. Определяет контракт для всех конкретных реализаций (TPNumber, TFrac, TComplex) через набор абстрактных методов: арифметические операции (Add, Subtract, Multiply, Divide), функции (Square, Inverse), проверка на ноль (IsZero), создание копии (Copy) и преобразование в/из строки (ToString, FromString). Реализует интерфейс ICloneable с реализацией Clone через вызов Copy(), что гарантирует единообразное клонирование для всех наследников.
3. **PNumber.cs** – модуль p -ичных чисел. Содержит класс TPNumber, наследующий от TANumber и представляющий действительное число в системе счисления с основанием от 2 до 16. Класс инкапсулирует значение числа в десятичном виде (`_value`), основание (`_base`) и точность представления (`_precision`). Предоставляет конструкторы для создания числа из вещественного значения, строки с указанием основания, а также методы для выполнения всех арифметических операций и функций над двумя TPNumber. Ключевой особенностью является проверка на совпадение оснований при выполнении операций над двумя p -ичными числами и явный метод ChangeBase для преобразования числа в другую систему счисления.
4. **TFrac.cs** – модуль обыкновенных дробей. Содержит класс TFrac, наследующий от TANumber и представляющий рациональное число в виде обыкновенной дроби. Хранит числитель (`_num`) и знаменатель (`_den`) в виде long. Обеспечивает автоматическую нормализацию дроби (сокращение на НОД и приведение знака к знаменателю) при создании и выполнении операций. Реализует все арифметические операции и функции в соответствии с правилами математики дробей. Ключевым поведением является представление целого числа в виде строки без знаменателя, если он равен 1.
5. **TComplex.cs** – модуль комплексных чисел. Содержит класс TComplex, наследующий от TANumber и представляющий комплексное число с действительной (`_re`) и мнимой

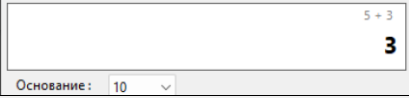
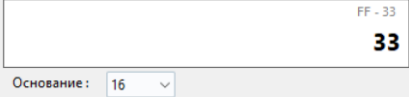
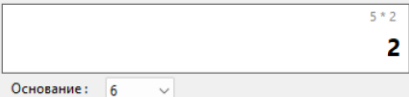
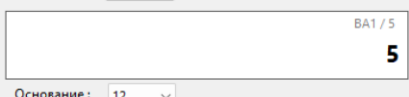
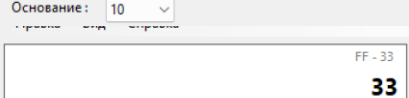
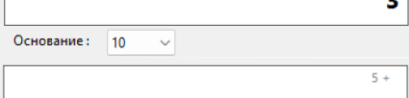
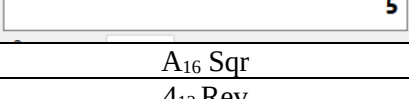
- (`_im`) частями в формате `double`. Реализует гибкий парсинг строки (`FromString`), поддерживающий форматы "`a+bi`", "`a-bi`", "`bi`", "`x+iy`", а также просто "`i`". Предоставляет методы для удобного форматирования вывода, включая специальную логику для случаев, когда действительная или мнимая часть равны нулю или единице. Все арифметические операции и функции реализованы строго по формулам для комплексных чисел.
6. **Editor.cs** – модуль редактора чисел. Реализует класс `Editor`, который отвечает за посимвольный ввод, хранение и редактирование строкового представления чисел для всех режимов калькуляции. Модуль хранит текущее число в `StringBuilder` и учитывает текущий режим (`_mode`) и основание (`_base`). Предоставляет методы для добавления цифр с учетом допустимости в текущей системе счисления, добавления знака, разделителя (для дробей — это `"/"`, для комплексных и `p`-ичных — `"."`), а также мнимой единицы.
 7. **Processor.cs** – модуль процессора операций. Содержит обобщённый класс `Processor<T>`, параметризованный типом `T`, который реализует вычислительное ядро программы. Инкапсулирует левый операнд/результат (`_lopRes`), правый операнд (`_rop`) и текущую операцию (`_operation`). Предоставляет методы для установки операндов и операции, выполнения двухоперандной операции (`ExecuteOperation`) и однооперандной функции (`ExecuteFunction`). После выполнения операции результат автоматически становится левым операндом, что позволяет строить цепочки вычислений. Использует динамическое связывание через полиморфизм для работы с разными типами чисел `TANumber`.
 8. **Memory.cs** – модуль памяти калькулятора. Содержит обобщённый класс `Memory<T>` для хранения одного числа произвольного типа `T`. Реализует конечный автомат из двух состояний (`Off/On`). Предоставляет полный набор операций для работы с памятью: `Clear` (сброс памяти в состояние `Off`), `Store` (сохранение значения и перевод в `On`), `Retrieve` (извлечение копии значения без изменения состояния) и `Add` (прибавление значения к хранимому числу с использованием динамического связывания через `dynamic`). Метод `GetString` возвращает "`M`" или пустую строку для отображения индикатора памяти в UI.
 9. **CalculatorControl.cs** – модуль управления логикой калькулятора. Выступает в роли контроллера приложения, координируя работу редактора, процессора, памяти и истории. Содержит реализацию конечного автомата на основе перечисления `CalculatorState`. Предоставляет центральный метод `ExecuteCommand`, который диспетчеризует 25+ команд пользователя (`CalculatorCommand`) в соответствующие обработчики. В методах-обработчиках реализована логика всех прецедентов: ввод цифр с проверкой основания, обработка унарного/бинарного минуса, выполнение отложенной операции, повтор операции по нажатию `"="`, вычисление функций, работа с памятью, буфером обмена и комплексной единицей. Модуль также управляет формированием истории вычислений и строки выражения через `_expressionParts`. Активно использует события `DisplayUpdated`, `MemoryStateUpdated`, `BaseChanged` и `ModeChanged` для уведомления UI об изменениях.
 10. **MainForm.cs** – модуль, реализующий главное окно приложения (класс `MainForm`, наследник `Form`). Отвечает за создание и размещение всех визуальных компонентов интерфейса: текстового поля для отображения числа (`displayTextBox`), метки состояния памяти (`memoryLabel`), метки текущего выражения (`expressionLabel`), выпадающего списка для выбора основания (`baseComboBox`), меню (Правка, Вид, Справка) и панели с кнопками (`TableLayoutPanel`). Содержит методы для динамического создания

кнопок (AddButton, AddDigitButton, AddHexButton), обработки нажатий клавиш (Form_KeyDown, Form_KeyPress), обновления состояния кнопок при смене основания (UpdateDigitButtonsState). Обеспечивает взаимодействие пользователя с калькулятором через мышь и клавиатуру, а также работу с буфером обмена через меню.

11. HistoryForm.cs – модуль, реализующий окно истории вычислений (класс HistoryForm, наследник Form). Отвечает за отображение списка выполненных операций в виде ListBox. Содержит кнопки «Очистить» для удаления всей истории и «Заккрыть» для закрытия окна. Предоставляет метод RefreshHistory для обновления отображаемого списка при добавлении новых записей или очистке истории. Записи отображаются в обратном хронологическом порядке (сначала новые).

12.

4. Тестирование:

Входные данные	Результат	Примечание
	[08:00:44] $5 + 3 = 8$ (осн. 10)	Сложение чисел в 10-ичной сс
	[08:01:16] $FF - 33 = CC$ (осн. 16)	Вычитание чисел в 16-ичной сс
	[08:02:00] $5 * 2 = 14$ (осн. 6)	Умножение чисел в 6-ичной сс
	[08:02:39] $BA1 / 5 = 245$ (осн. 12)	Деление чисел в 12-ичной сс
	[08:02:54] $8 * 8 = 64$ (осн. 10)	Операция с одним операндом
	[08:03:08] $CC - 33 = 99$ (осн. 16) [08:03:07] $FF - 33 = CC$ (осн. 16)	Повторное выполнение
	[08:04:14] $5 + 5 = 10$ (осн. 10) [08:04:12] $2 + 3 = 5$ (осн. 10)	Операция над результатом
$A_{16} \text{ Sqr}$	[08:04:24] $\text{Sqr}(A) = 64$ (осн. 16)	Возведение в квадрат
4_{12} Rev	[08:04:42] $\text{Rev}(4) = 0,3$ (осн. 12)	Операция $1/x$
	[08:05:01] $A + B * C = 11C$ (осн. 15)	Сложное выражение
		Перевод числа в другую сс при её выборе
	[16:47:12] $1/2 + 1/3 = 5/6$	Сложение дробей
	[16:47:58] $2/3 - 1/6 = 1/2$	Вычитание дробей

3/4 * 2/5 Режим: Простые дроби Основание:	[16:48:11] $3/4 * 2/5 = 3/10$	Умножение дробей
Sqr(2/3)	[16:49:17] $\text{sqr}(2/3) = 4/9$	Возведение дроби в квадрат
Rev(1/4)	[16:50:06] $1/(1/4) = 4$	Операция $1/x$
1 + 2i + 3 + 4i Режим: Комплексные чи Основание:	[16:58:25] $1 + 2i + 3 + 4i = 4+6i$	Сложение комплексных чисел
5 + 6i - 2 + 3i Режим: Комплексные чи Основание:	[16:58:55] $5 + 6i - 2 + 3i = 3+9i$	Вычитание комплексных чисел

5. Вывод:

В ходе выполнения лабораторной работы был проведено проектирование программ в технологии «абстрактных типов данных», реализованы абстрактные типы данных с помощью классов C#, были использованы библиотеки визуальных компонентов для построения интерфейсов, а также создан инсталлятор для установки калькулятора на другие компьютеры с ОС Windows.

6. Приложение

Код представлен на GitHub: <https://github.com/1n6ty/NSTU-lab/tree/TRPO/lab-4>